Model querying using AQL

A model may be queried using a combination of GraphQL and AQL (Acceleo query language https://eclipse.dev/acceleo/documentation/) syntax. The GraphQL wraps the call to the web server and the embedded AQL queries any UML or Ecore-based models. Its syntax is similar to OCL (Object Constraint Language), with some simplifications.

The AQL querying facilies are not integrated with the modeler interface, but an URL is available to start querying.

The URL is http://localhost:8080/graphiql/index.html

The overall structure of a query is shown below. It queries all elements of the model that have 'Monitoring' as part of the name. The *editingContext* parameter is the ID of the root project, which is available in the project URL.

The example projet URL is : http://localhost:8080/projects/0cdf3930-0ea6-4560-aa9f-7895d48a6908/edit. The editingContext is *0cdf3930-0ea6-4560-aa9f-7895d48a6908*. The complete GraphQL schema is available in this link: http://localhost:8080/voyager/index.html. However, for most part of the queries, the structure presented below is enough, since the complexity of the query is in the AQL part.

```
query Viewer {
  viewer {
    editingContext(editingContextId: "0cdf3930-0ea6-4560-aa9f-7895d48a6908") {
      queryBasedObject(query: "aql:editingContext.allContents()->select(e |
e.eClass().name='Class' and e.name.contains('Monitoring')) ") {
        queryBasedString(query: "aql:self")
      }
    }
  }
}
```

The output is a JSON following the PapyrusWeb GraphQL schema. The array inside *queryBasedString* contains the model element values.

```
{
  "data": {
    "viewer": {
      "editingContext": {
        "queryBasedObject": {
          "queryBasedString": "[org.eclipse.uml2.uml.internal.impl.ClassImpl@8c199ea
(name: HeartMonitoring, visibility: <unset>) (isLeaf: false, isAbstract: false,
isFinalSpecialization: false) (isActive: false)]"
        }
      }
    }
  }
```

```
}
```

Consider the more complex example shown below. It returns all classes that have the *Sensor* stereotype applied. This example shown the possibility of doing structural verification of a given IoMT architecture, for instance, by checking the existence of specific *Assets* or *IoMTComponents*

```
query Viewer {
  viewer {
    editingContext(editingContextId: "0cdf3930-0ea6-4560-aa9f-7895d48a6908") {
      queryBasedObject(query: "aql:editingContext.allContents()->select(e |
e.getAppliedStereotypes()->exists( st | st.name = 'Sensor') ") {
        queryBasedString(query: "aql:self")
      }
    }
  }
}
```

The method *getStereotypeApplications()* returns all the stereotype applications for a given element and its corresponding values.

```
query Viewer {
  viewer {
    editingContext(editingContextId: "0cdf3930-0ea6-4560-aa9f-7895d48a6908") {
      queryBasedObject(query: "aql:editingContext.allContents()->select(e |
e.getAppliedStereotypes()->exists( st | st.name = 'Sensor') ") {
        queryBasedString(query: "aql:self.getStereotypeApplications()")
      }
    }
  }
}
```

More sample queries are available here Query samples